

第一讲绪论

一、算法的时间复杂度分析

定义：对于程序大致运行时间的描述

1.单循环

解题方式：写出次数和条件值即可

```
int i=0;
while(i*i*i<=n)
{
    i++;
}
```

次数	0	1	2	...	k
条件	$0^3 \leq n$	$1^3 \leq n$	$2^3 \leq n$	...	$k^3 \leq n$

$$k^3 < n$$

```
int x=0;
while(n>=(x+1)*x+1)
{
    i++;
}
```

次数	0	1	2	...	k
条件	1	2^2	3^2	...	$(k+1)^2$

$$(k+1)^2 = n$$

2.双循环

当外层不影响内层的情况，当成两次单循环即可：

```
int x=0;
for(int k=1;k<=n;k*=2)
{
    for(int j=1;j<=n;j++)
    {
        x++;
    }
}
```

算出两个复杂度内外循环相乘

当外层会影响内层时候：

```
int m=0,i,j
for(int i=0;i<=n;i++)
{
    for(int j=1;j<=2*i;j++)
    {
        m++
    }
}
```

$T(n) = \sum_{i=1}^n \sum_{j=1}^{2i} 1$ 原理如下(其实就是吧i,j,内层执行次数写出来然后对齐求和)：

外层 i	内层 j 范围	内层执行次数	m 的累加过程
0	1 到 0 (不成立)	0	m = 0
1	1 到 2	2	m = 0 + 2
2	1 到 4	4	m = 0 + 2 + 4
...	...	...	...
n	1 到 2n	2n	m = 2 + 4 + ... + 2n

扩展：

```
int m=0;
for(int i=0;i<=n;i++)
{
    for(int j=1;j<=2*i;j++)
    {
        for(int k=1;k<=j;k++)
            m++;
    }
}
```

$$T(n) = \sum_{i=1}^n \sum_{j=1}^{2i} \sum_{k=1}^j 1$$

二、算法的空间复杂度分析

1. 固定空间型（常数阶）

这种题型中，无论输入数据规模多大，算法分配的变量个数都是固定的。

特征：只定义了几个简单的临时变量（如

i

,

j

,

temp

,

sum

)。

示例：

```
int sum = 0;
for(int i = 0; i < n; i++) {
    sum += i;
}
```

解题：空间复杂度为 $O(1)$ 。

2. 线性空间型（线性阶）

算法内部定义了与输入规模 成正比的存储结构。

特征：定义了数组、链表、栈或队列，且其长度取决于 。

示例：

```
int* arr = new int[n]; // 分配了n个大小的空间
for(int i = 0; i < n; i++) arr[i] = i;
```

解题：空间复杂度为 。如果定义的是二维数组

```
new int[n][n]
```

，则为 $O(n)$ 。

3. 递归空间型

这是考试中最容易出错的类型。递归不仅看代码里有没有定义变量，还要看**递归调用的深度**（即系统栈占用的空间）。

特征：函数自我调用。

计算公式：空间复杂度 = 递归深度*每次递归调用的局部变量空间。

示例 A（普通递归）：

```
void print(int n) {
    if (n <= 0) return;
    print(n - 1);
}
```

递归深度为n，每次调用只占用常数空间，故为O(n)。

解题方法归纳

在分析空间复杂度时，可以套用以下步骤：

步骤	操作方法
第一步：找存储变量	检查代码中是否有 <pre>new</pre> , <pre>malloc</pre> 或直接定义的数组、集合。如果有，看其大小是否随 变化。
第二步：判定递归深度	如果有递归，画出递归树，确定从根到叶子的最长路径长度（深度）。
第三步：忽略辅助开销	算法的空间复杂度只考虑 为实现算法而额外开辟的空间 ，输入数据本身占用的空间不计入。
第四步：取最高阶	和时间复杂度一样，保留最高阶项，忽略常数系数。

第三讲栈和队列

一、出栈可能计算公式

$\frac{1}{n+1}C_{2n}^n$ 又叫做卡特兰数

二、非循环队列和循环队列

特性	非循环队列（普通顺序队列）	循环队列 (推荐)
空间利用率	低。会产生“假溢出”，出队后的空间无法重复利用。	高。通过数学取模逻辑，使空间首尾相接循环利用。
指针更新	<pre>ptr++</pre>	<pre>ptr = (ptr + 1) % MaxSize</pre>
队满判定	<pre>rear == MaxSize - 1</pre>	<pre>(rear + 1) % MaxSize == front</pre>
队空判定	<pre>front == rear</pre>	<pre>front == rear</pre>

第四讲串

一、子串

个数计算公式 $\frac{n(n+1)}{2} + 1$

原理：空 1，本身 1，接下来就是 2， 3， 4....n-1(随长度递减)

二、串的模式匹配

1. BF算法

将目标串 S 的第一个字符与模式串 T 的第一个字符进行比较 2。若相等，则继续比较后续字符；若不等，则目标串 S 回退到本趟开始字符的下一个字符，模式串 T 回退到开头，重新开始下一趟匹配 3。

```

int BF(string s, string t) {
    int i = 0, j = 0; // i 指向目标串 s, j 指向模式串 t

    // 当两串均未遍历完时进行循环
    while (i < s.length() && j < t.length()) {
        if (s[i] == t[j]) { // 比较的两个字符相同时
            i++;           // s 和 t 指针同时后移
            j++;
        }
        else {             // 比较的两个字符不同时（失配）
            i = i - j + 1; // i 回退到本趟开始位置的下一个字符
            j = 0;         // j 移动到模式串 t 的开头
        }
    }

    // 如果 j 遍历完了模式串，说明匹配成功
    if (j >= t.length())
        return i - t.length(); // 返回模式串在目标串中的起始位置
    else
        return -1;             // 匹配不成功，返回 -1
}

```

理解 $i = i - j + 1$ 其中 $(i - j)$ 是计算出当前这一趟匹配是从主串的哪个位置开始的。如果你已经在这一趟匹配中对比了 j 个字符，那么主串的起始点就是当前位置 i 减去移动的距离 j , 因为 j 会被置为 0。

2.KMP算法

在BF算法中，一旦失配，主串指针 i 会退回并重新开始，这造成了大量重复比较。KMP认为：已经匹配成功的那些字符包含了重要的信息。如果模式串的前缀和失配位置之前的后缀有重合部分，我们就可以直接把模式串“滑”过去，跳过肯定不匹配的步骤。于是该操作的核心就是维护了一个next数组，我们先来看看什么是next数组

next数组：next[j]就是说当模式串中的第 j 个字符发生失配时，模式串应该回退到的新位置，不像BF算法直接回退到开头了

知道next数组之后我们来聊一下他的**求解逻辑**：

对于模式串 t ， $\text{next}[j]$ 的值取决于子串 $t[0 \dots j-1]$ 中最长的相等前后缀的长度。特殊规定 $\text{next}[0] = -1$ ：表示第 0 个字符就匹配失败，主串指针 i 需后移。
 $\text{next}[1] = 0$ ：表示第 1 个字符匹配失败，只能回退到第 0 个字符重新比

我们不是独立计算每个 $\text{next}[j]$ ，而是利用 $\text{next}[j]$ 来推导 $\text{next}[j+1]$ ，假设已知 $\text{next}[j] = k$ ，这意味着 $t[0 \dots k-1] = t[j-k \dots j-1]$

于是我们就考虑如下两种情况：

情况 A：字符匹配成功如果 $t[j] == t[k]$ ：既然前面的部分已经相等，现在末尾又加了一个相等的字符，那么相等的前后缀长度就增加了 **结论： $\text{next}[j+1] = k + 1$** 。

情况 B：字符匹配失败如果 $t[j] \neq t[k]$ ：说明当前的公共前后缀无法延续。我们需要找一个更短的公共前后缀。操作：令 $k = \text{next}[k]$ 。逻辑：这相当于把模式串相对于自己进行了一次 KMP 跳转，一直回退到某个能匹配上 $t[j]$ 的位置，或者回退到 -1 为止。

```
void GetNext(string t, int* next) {
    int j = 0;        // j 指向当前正在计算的位置
    int k = -1;       // k 代表最长公共前后缀的长度
    next[0] = -1;     // 固定初始值

    while (j < t.length() - 1) { // 遍历模式串
        // k == -1 代表需要重新开始，或者当前字符匹配成功
        if (k == -1 || t[j] == t[k]) {
            j++;
            k++;
            next[j] = k; // 成功递推: next[j+1] = k+1
        }
        else {
            k = next[k]; // 失败回退: 寻找更短的公共前后缀 [
        }
    }
}
```

以模式串

```
t = "abcac"
```

为例，其

数组的推导过程如下：

j	$t[j]$	$t[j]$ 前面的子串	相等前后缀情况	next[j]
0	a	无	无	-1
1	b	"a"	无	0
2	c	"ab"	无	0
3	a	"abc"	无	0
4	c	"abca"	"a" == "a" (长度为1)	1

```
int KMP(string s, string t) {
    int n = s.length(), m = t.length();
    int* next = new int[m];
    GetNext(t, next); // 1. 预计算局部匹配信息 next 数组

    int i = 0, j = 0; // i 指向主串, j 指向模式串
    while (i < n && j < m) { // 两串均未遍历完时循环
        // j == -1 (对应next[0]) 或当前字符匹配成功
        if (j == -1 || s[i] == t[j]) {
            i++;
            j++; // 指针双双后移
        }
        else {
            j = next[j]; // 失配: i 不变, j 根据 next 数组回退
        }
    }

    if (j >= m) return i - m; // 匹配成功: 返回起始位置
    else return -1;           // 匹配失败: 返回 -1
}
```

3.改进的KMP算法

其实就是改变了next数组的求法，它消除了模式串中连续重复字符带来的无效匹配尝试，使跳转更加彻底。

```
void GetNextval(string t, int* nextval) {
    int j = 0, k = -1;
    nextval[0] = -1; // 初始规定

    while (j < t.length() - 1) {
        if (k == -1 || t[j] == t[k]) { // 字符匹配成功
            j++; k++;
            if (t[j] != t[k])
                nextval[j] = k; // 情况1: 字符不同, 正常记录
            else
                nextval[j] = nextval[k]; // 情况2: 字符相同, 直接继承
        }
        else {
            k = nextval[k]; // 字符不匹配, k 回退
        }
    }
}
```

邪修求法(选择题):



写出next数组后，全加1从1->2，得到next2

之后写出每个数字对应位数的字母

再接下来和原来字符串比对，如果不同，则照搬next2数组数值，nextval[0]=0。

最后开始写nextval数组，根据next2数值，去找nextval已有结果比如

```
nextval[k]=nextval[next2[k]-1]
```

第五讲数组和稀疏矩阵

按行优先存储

在 m 行 n 列的二维数组 $A_{m \times n}$ 中

假设每个元素占 k 个存储单元， $LOC(a_0, 0)$ 表示 $a_0, 0$ 元素的存储地址。对于元素 $a_{i,j}$ 元素 $a_{i,j}$ 前面有从第 0 到 $i-1$ 共 i 个完整行，每行有 n 个元素，总计 $i \times n$ 个元素 3。

$$LOC(a_{i,j}) = LOC(a_{00}) + (i \times n + j) \times k$$

按列优先存储

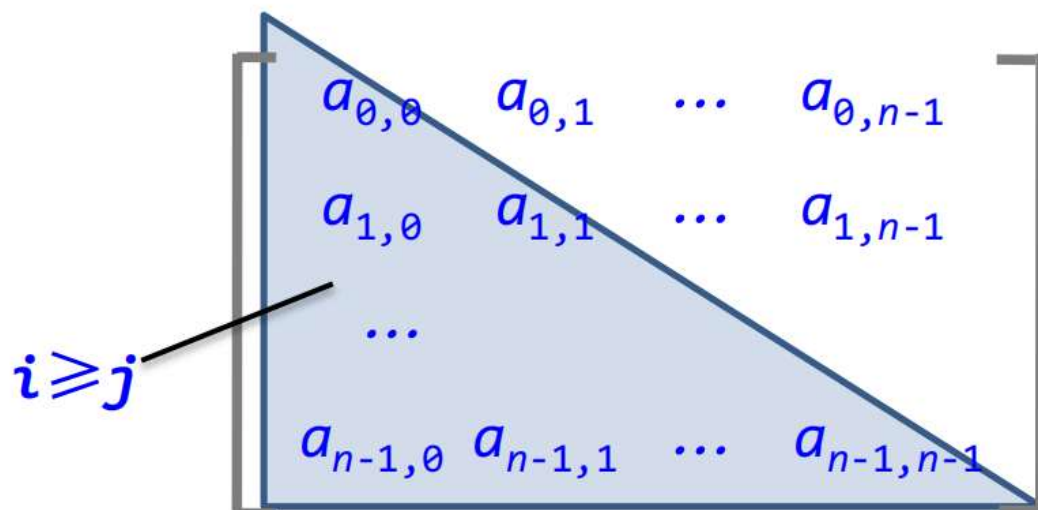
$$LOC(a_{i,j}) = LOC(a_{0,0}) + (j \times m + i) \times k$$

用如上公式计算的时候要注意**起始位置**

特殊矩阵的压缩存储

压缩存储：若多个数据元素的值都相同，则只分配一个元素值的存储空间，且零元素不占存储空间(对称矩阵，对角矩阵，三角矩阵，稀疏矩阵等)

1. 对称矩阵的压缩存储



若一个 n 阶方阵 A 的元素满足 $a_{i,j} = a_{j,i}$ ($0 \leq i, j \leq n-1$)，则称其为 n 阶对称矩阵。

假设一维数组 B 的下标 k 从 0 开始 (即 $B[0..n(n+1)/2-1]$)，则矩阵元素 $a_{i,j}$ 在 B 中的位置 k 为：

$$k = \begin{cases} \frac{i(i+1)}{2} + j, & \text{当 } i \geq j \text{ 时 (下三角元素)} \\ \frac{j(j+1)}{2} + i, & \text{当 } i < j \text{ 时 (上三角元素)} \end{cases}$$

公式推导 ($i \geq j$ 情况):

在 $a_{i,j}$ 所在的第 i 行之前，共有 i 行 (从第 0 行到第 $i-1$ 行)

第 0 行存 1 个元素，第 1 行存 2 个，...，第 $i-1$ 行存 i 个。前 i 行总元素个数为 $1 + 2 + \dots + i = \frac{i(i+1)}{2}$

在第 i 行中， $a_{i,j}$ 之前有 j 个元素 (下标从 0 到 $j-1$)。

因此， $a_{i,j}$ 是第 $\frac{i(i+1)}{2} + j$ 个元素

随机存取：即使经过压缩，通过下标变换公式仍可在 $O(1)$ 时间内定位元素，保留了随机存取特性

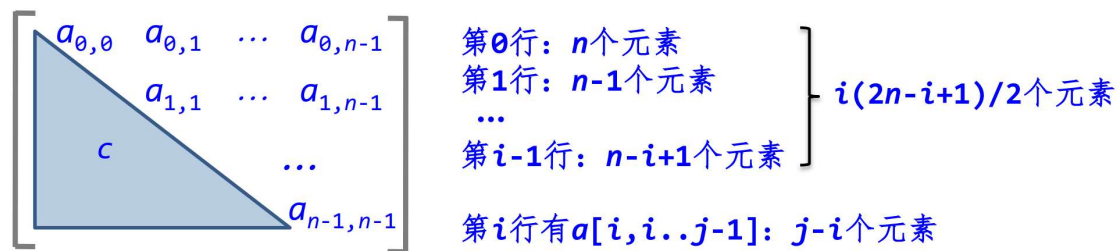
应用：在程序中通过

```
getk(i, j)
```

函数实现二维坐标到一维下标的转换，从而直接对压缩后的数组进行运算 (如矩阵乘法)

2. 上三角矩阵的压缩存储

它的下三角(不包括主角线)中的元素均为常数 c 。



$$k = \begin{cases} \frac{i(2n-i+1)}{2} + j - i & \text{当 } i \leq j \text{ 时} \\ \frac{n(n+1)}{2} & \text{当 } i > j \text{ 时} \end{cases}$$

存放常量 c

下三角矩阵

AI识图

$$\begin{bmatrix} a_{0,0} & & & \\ a_{1,0} & a_{1,1} & & \\ \cdots & & \ddots & \\ a_{n-1,0} & a_{n-1,1} & \cdots & a_{n-1,n-1} \end{bmatrix}$$

$i \geq j$

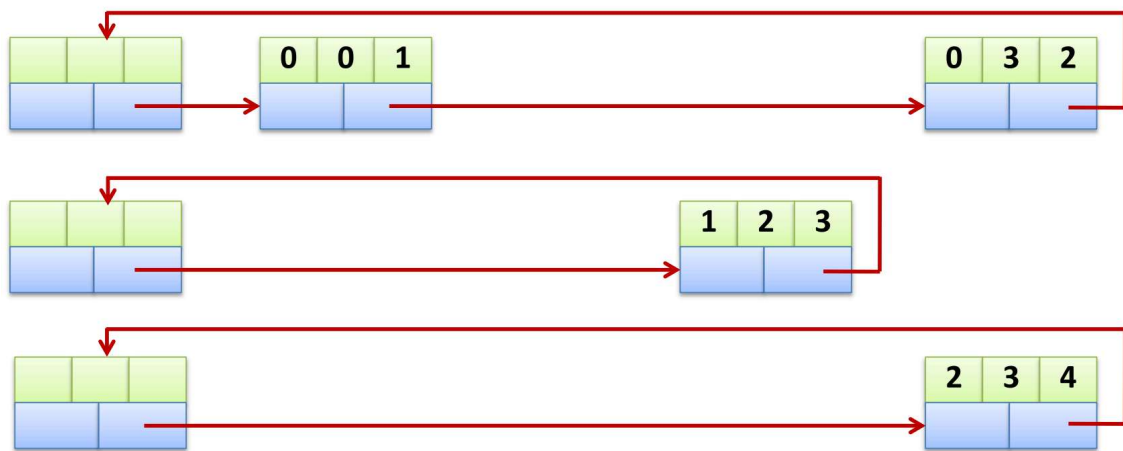
$$k = \begin{cases} \frac{i(i+1)}{2} + j & \text{当 } i \geq j \text{ 时} \\ \frac{n(n+1)}{2} & \text{当 } i < j \text{ 时} \end{cases}$$

存放一个常量 c

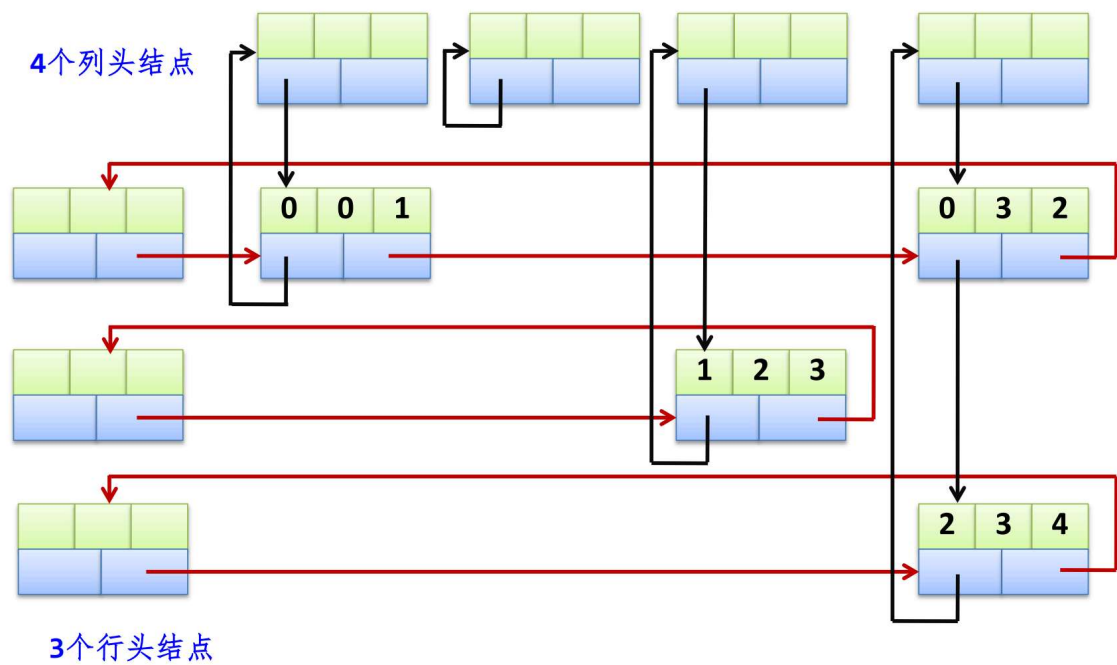
3. 稀疏矩阵的压缩存储

三元组表示法： 稀疏矩阵中的每一个非零元素需由一个三元组 (i, j, a_{ij})

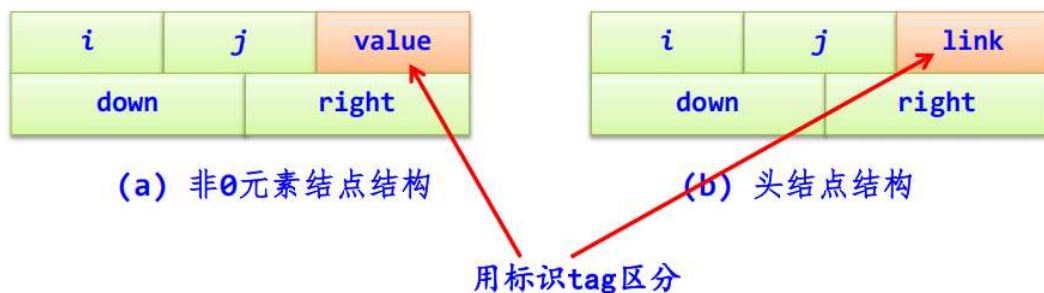
十字链表示法： 每行的所有结点链起来构成一个带行头结点的循环单链表。以 $h[i]$ ($0 \leq i \leq m-1$) 作为第 i 行的头结点



每列的所有结点链起来构成一个带列头结点的循环单链表。以 $h[i]$ ($0 \leq i \leq m-1$) 作为第 i 列的头结点



结点设计方法：



第七讲树和二叉树

与树相关的数学性质(结点数，边数，度数，高度)

$$\text{分支数} = n - 1$$

$$n = \sum(\text{所有节点的度}) + 1 \text{ (这里是出度)}$$

$$\text{分支数} = \text{度之和}$$

$$\text{度为} m \text{的树中第} i \text{层上至多} m^{i-1} \text{个结点}$$

$$\text{高度为} h \text{的} m \text{次树至多有} \frac{m^h - 1}{m - 1} \text{个结}$$

$$\text{推导：因为第} i \text{层最多有} m^{i-1} \text{个结点，于是就是} \sum_{i=1}^h m^{i-1} = m^0 + m^1 + m^2 + \dots + m^{h-1} = \frac{1 \times (m^h - 1)}{m - 1} = \frac{m^h - 1}{m - 1}$$

$$\text{具有} n \text{个结点的} m \text{次数的最小高度为} \log_m(n(m - 1) + 1)$$

$$\text{推导：设具有} n \text{个结点的} m \text{次树的最小高度为} h, \text{若在该树中前} h-1 \text{层都是满的那么第} h \text{层可能满也可能不满于是} \frac{m^{h-1} - 1}{m - 1} \leq n < \frac{m^h - 1}{m - 1} \text{ 求解得到 } \log_m(n(m - 1) + 1) \leq h < \log_m(n(m - 1) + 1) + 1$$

因 h 只能取整数,所以 $h = \log_m(n(m-1) + 1)$

二叉树相关性质

满二叉树： 如果所有分支结点都有左孩子结点和右孩子结点，并且叶子结点都集中在二叉树的最下一层

叶子结点都在最下一层

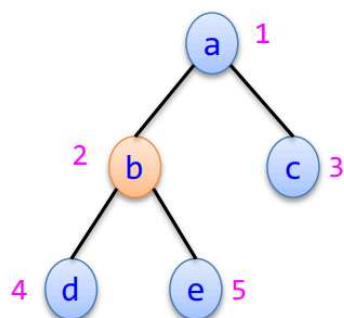
只有度为0和度为2的结点

含 n 个结点的满二叉树的高度为 $\log_2(n+1)$

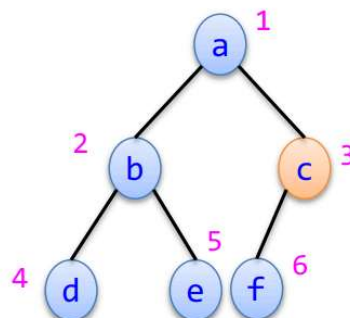
完全二叉树： 若二叉树中最多只有最下面两层的结点的度数可以小于2，并且最下面一层的叶子结点都依次排列在该层最左边的位置上

当完全二叉树按**层序编号**后有如下性质(假设**根节点编号为1**):

最后一个分支结点编号为 $\frac{n}{2}$



$n=5 \Rightarrow n_1=0$, 最后
分支结点为 $n/2=2$



$n=6 \Rightarrow n_1=1$, 最后
分支结点为 $n/2=3$

若 n 为奇数，则 $n_1 = 0$,每个分支结点都是双分支结点,相反若 n 为偶数，则只有一个单分支结点

若编号为 i 的结点有左孩子结点，则左孩子结点的编号为 $2i$ ；若编号为 i 的结点有右孩子结点，则右孩子结点的编号为 $2i+1$

若编号为 i 的结点有左兄弟结点，左兄弟结点的编号为 $i-1$ ，若有右兄弟结点，右兄弟结点的编号为 $i+1$

若编号为 i 的结点有双亲结点，其双亲结点的编号为 $\lceil i/2 \rceil$ 。

具有 n 个 ($n > 0$) 结点的完全二叉树的高度为 $\log_2(n+1)$ 或 $\log_2(n) + 1$

二叉树：

叶子结点（没有孩子的结点）的数量，永远比双分支结点（有两个孩子的结点）的数量多一个

推导：总结点数 $n = n_0 + n_1 + n_2$ 。 ,有因为总的度数=总分支数= $n-1$ ，于是

$$n_0 = n_2 + 1$$

非空二叉树上第 i 层上至多有 2^{i-1} 个结点

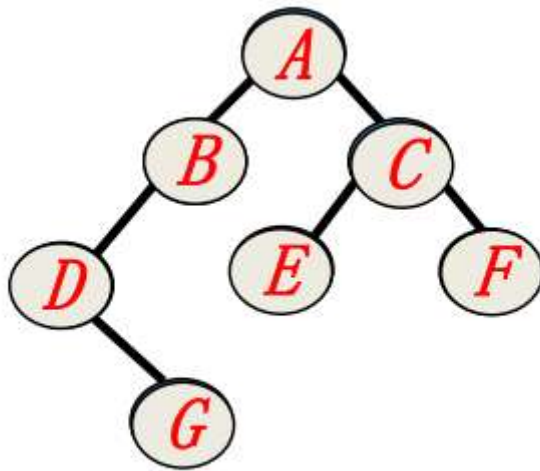
高度为 h 的二叉树至多有 $2^h - 1$ 个结点

二叉树的遍历

先序遍历:

```
void PreOrder11(BTNode* b) //被PreOrder函数调用
{
    if (b!=NULL)
    {
        cout << b->data; //访问根结点
        PreOrder11(b->lchild); //先序遍历左子树
        PreOrder11(b->rchild); //先序遍历右子树
    }
}

void PreOrder1(BTree& bt) //先序遍历的递归算法
{
    PreOrder11(bt.r);
}
```



访问顺序: ABDGCEF

中序遍历


```

void InOrder11(BTNode* b)
{
    if (b!=NULL)
    {
        InOrder11(b->lchild);
        cout << b->data;
        InOrder11(b->rchild);
    }
}

void InOrder1(BTree& bt)
{
    InOrder11(bt.r);
}

```

访问顺序：DGBAECF

后序遍历

```

void PostOrder11(BTNode* b)
{
    if (b!=NULL)
    {
        PostOrder11(b->lchild);
        PostOrder11(b->rchild);
        cout << b->data;
    }
}

void PostOrder1(BTree& bt)
{
    PostOrder11(bt.r);
}

```

访问顺序：GDBEFCA

层次遍历

```
void LevelOrder(BTree& bt) //二叉树的层次遍历
{
    BTreeNode* p;
    queue<BTreeNode*> qu; //定义一个队列
    qu.push(bt.r); //根结点r进队
    while (!qu.empty()) //队不空时循环
    {
        p=qu.front(); qu.pop(); //出队结点p
        cout << p->data; //访问结点p
        if (p->lchild!=NULL) //有左孩子时将其进队
            qu.push(p->lchild);
        if (p->rchild!=NULL) //有右孩子时将其进队
            qu.push(p->rchild);
    }
}
```